CSE 4141 · MICROPROCESSOR INTERFACING & MICROCONTROLLERS LABORATORY

Experiment 04 · I²C – Two-Wire Addressed Bus

I2C1 master and slave at 100 kHz, addressed transfer with hardware acknowledge

NUCLEO-F446RE × 2 STM32F446RET6, Cortex-M4	**I2C1 (APB1)** peripheral used	**100 kHz standard mode** line settings	**3 (SCL, SDA, GND) + 2 pull-ups** wires between boards	**Bare metal** CMSIS registers, no HAL, no CubeMX

1 Aim and specification

Two NUCLEO-F446RE boards talk to each other over **I2C** and nothing else. The board holding the **master** listing reads its blue B1 user button and measures how long it is held; the board holding the **slave** listing has no button and reacts only to what arrives on the I2C link, blinking its green LD2 in one of two clearly distinguishable patterns. After programming, both boards run from any 5 V USB supply with no PC attached.

1.1 Behavioural specification

Stimulus on the master	Measured press	Byte transmitted	Slave LD2 pattern	LED-to-LED spacing	Total
Short click	< 1000 ms	`'S'` (0x53)	3 blinks · 150 ms ON / 150 ms OFF	300 ms	0.9 s
Press and hold	≥ 1000 ms	`'L'` (0x4C)	3 blinks · 1000 ms ON / 1000 ms OFF	2000 ms	6.0 s

The long-press pattern is the point of the experiment: 1000 ms on plus 1000 ms off means the slave LED starts a blink every **2.0 s**, against 300 ms for a short click. Both numbers are the two literals in the slave's `blink()` call, so the timing can be retuned by editing one line. The master flashes its own LD2 for 100 ms each time it sends, so you can always see which board acted.

1.2 How short these programs are

Listing	Lines of code	What it does
Master	about 30	clock enables, four GPIO lines, peripheral init, one loop
Slave	about 25	clock enables, two GPIO lines, peripheral init, one loop

There is no interrupt handler, no clock-tree configuration, no acknowledgement protocol and no checksum. Everything that remains is something an examiner can point at and ask you to justify – which is exactly what you want when you are typing from memory.

2 Apparatus

Qty	Item	Notes
2	NUCLEO-F446RE development board	one typed with the master listing, one with the slave listing
1 set	Male–male jumper wires (Dupont)	keep them under about 20 cm
2	USB Mini-B cables	for programming, then for standalone power
1–2	USB power bank or 5 V phone charger	to run the boards away from the PC
2	4.7 kΩ resistors + small breadboard	I2C experiment only – mandatory bus pull-ups

3 Pin assignment

MCU pin	Function	Configuration	Morpho	Arduino	Used by
PB8	I2C1_SCL	AF4, **open drain**	CN10-3	D15	clock, master drives
PB9	I2C1_SDA	AF4, **open drain**	CN10-5	D14	bidirectional data
–	2 × 4.7 kΩ to +3V3	external resistors	CN7-16 (+3V3)	–	mandatory bus pull-ups
PC13	B1 user button	input + pull-up	CN7-23	–	master only
PA5	LD2 user LED	output	CN10-11	D13	both boards
GND	0 V reference	–	CN10-20	–	mandatory

Reading the headers
CN7 and CN10 are the two 2×19 “morpho” headers along the long edges. Pin 1 of each is the row nearest the ST-LINK part of the board; odd numbers are in the left column and even numbers in the right column when the board is seen from above with the USB connector upwards. Every pin name is printed on the underside of the PCB – when in doubt, find a pin by its *name*, not by counting rows.

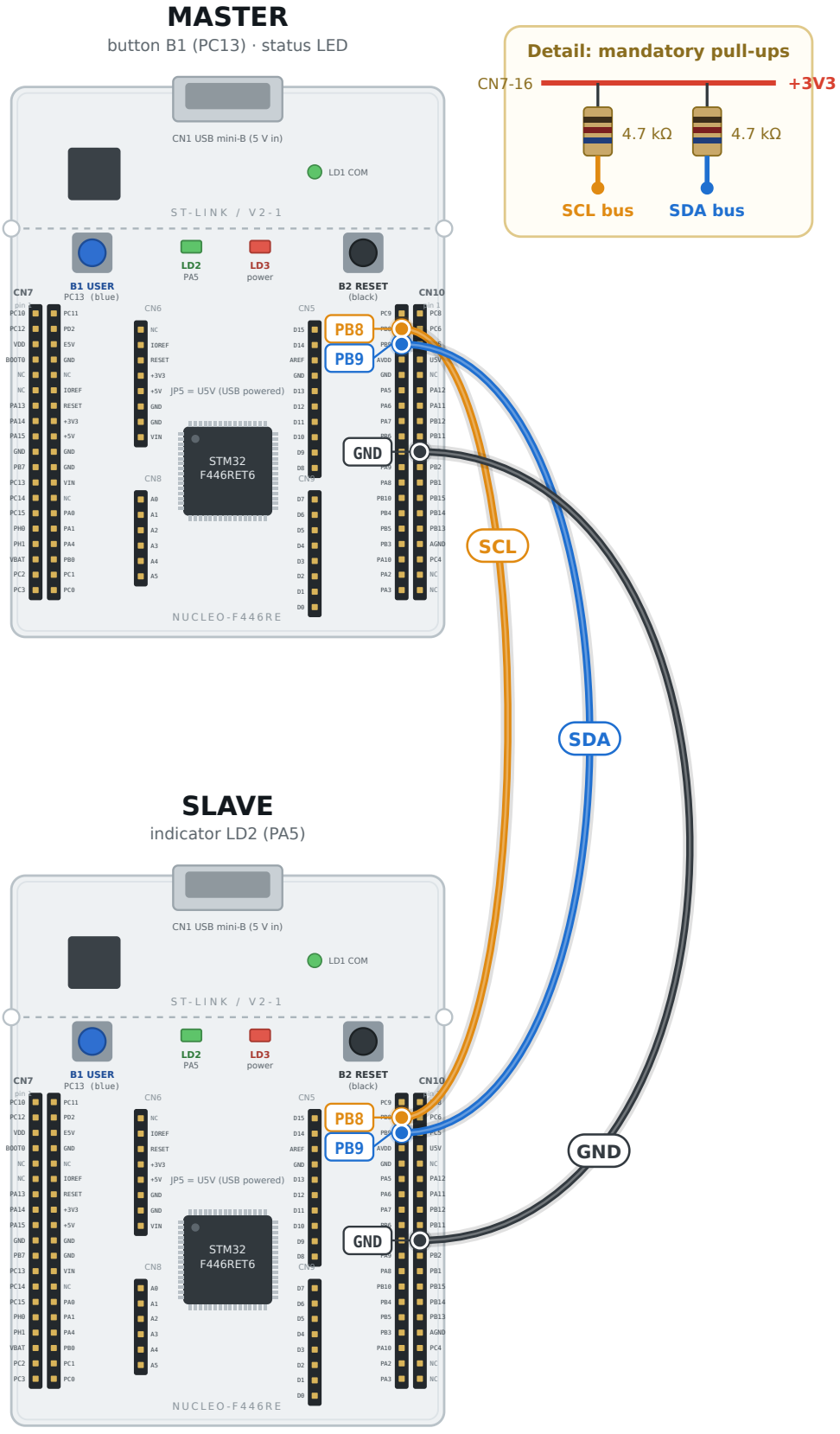
4 Circuit and wiring

Make every connection with the boards **unpowered**. The colours in the table are the ones used in Figure 1 on the next page; any colour works electrically, but keeping a convention makes a wiring fault obvious at a glance.

Signal	MASTER pin	SLAVE pin	Wire colour used	Topology
SCL	PB8 · CN10-3	PB8 · CN10-3	orange	common bus
SDA	PB9 · CN10-5	PB9 · CN10-5	blue	common bus
Ground	GND · CN10-20	GND · CN10-20	black	common
Pull-ups	4.7 kΩ SCL → +3V3	4.7 kΩ SDA → +3V3	–	one pair for the whole bus

Experiment 4 · I2C1 – board-to-board wiring

Top view, USB connectors upwards. Pin names are silk-screened on the underside of each board.



SCL and SDA are common bus wires, not crossed. Both pins are open drain, so the pull-ups create every logi
One resistor pair for the whole bus is enough – fit it on a breadboard.
PB8 = CN10-3 = D15 | PB9 = CN10-5 = D14 | +3V3 = CN7-16 | GND = CN10-20.

Figure 1 – Physical wiring between the two boards, drawn to the real board outline. The coloured rings mark the pads that carry a wire.

5 Theory of operation

> **Without the pull-up resistors nothing works**
> Both I2C pins are set to **open drain**: the output transistor can only pull the line down to 0 V, never up. The high level is produced entirely by the external resistors. The NUCLEO-F446RE has none fitted on PB8/PB9, so two 4.7 kΩ resistors to +3V3 are part of the circuit, not an optional extra.

I2C replaces the chip-select wire of SPI with an **address** carried on the bus itself. Every transfer opens with a START condition (SDA falls while SCL is high), then seven address bits and a direction bit. The addressed slave answers by pulling SDA low for one clock – the acknowledge – a hardware handshake that neither SPI nor UART provides. The slave here answers to `0x28`, which appears on the wire as `0x28 << 1 = 0x50` with the read/write bit in bit 0.

Three numbers instead of one baud-rate register

Field	Value	Meaning
`CR2` (FREQ)	16	the APB1 clock in MHz, so the peripheral can work out its own timings
`CCR`	80	standard mode: CCR = f_{PCLK1} / (2 × f_{SCL}) = 16 000 000 / 200 000 – the master needs this, the slave does not
`TRISE`	17	maximum rise time 1000 ns in PCLK1 periods, plus one: 1000 / 62.5 + 1

The two STM32 quirks that catch everybody

- **Clearing ADDR.** The address-matched flag is cleared by reading `SR1` and *then* `SR2`, in that order – that is what the bare `(void)I2C1->SR2;` line is for. Skip it and the peripheral holds SCL low and the bus freezes. Reading `SR2` also reveals, through the `TRA` bit, whether the master wants to write or read.
- **OAR1 bit 14 and the order of PE and ACK.** Bit 14 of `OAR1` is reserved but must be written as 1. And `CR1.ACK` has to be set *after* `PE`, because enabling the peripheral clears it – a slave that silently NACKs its own address is almost always this mistake.

Finally, `STOPF` is cleared by reading `SR1` and then writing `CR1`, which is the last pair of lines in the slave loop. Without it the slave services exactly one transfer and then goes deaf.

The `WAIT()` macro puts a bounded spin on each flag, so a missing or unpowered slave makes the master give up after a few milliseconds instead of hanging forever – worth having when you are demonstrating under time pressure.

> **Why there is no clock-configuration code**
> After reset the STM32F446 runs from its internal 16 MHz RC oscillator (HSI) with every bus prescaler at /1, and CubeIDE's `SystemInit()` does not change that – it only enables the FPU. So **SYSCLK = HCLK = PCLK1 = PCLK2 = 16 MHz** without a single line of setup. That single fact is where the two constants in the listing come from: `16000` in `delay_ms()` (16 MHz / 1 kHz) and the peripheral timing value in the init.
>
> Do not paste a `SystemClock_Config()` from a HAL example into these projects. If you raise SYSCLK, the delays and the baud rate move with it.

5.2 The delay function, without an interrupt

All eight listings share the same six-line timer. SysTick is a 24-bit down-counter inside the Cortex-M4 core, so it needs no RCC clock enable and no NVIC setup:

```
SysTick->LOAD = 16000 - 1;   /* reload = 1 ms at 16 MHz  */
SysTick->VAL  = 0;           /* clear the counter        */
SysTick->CTRL = 5;           /* bit0 ENABLE, bit2 CLKSOURCE = core clock */
```

```
while (ms--) while (!(SysTick->CTRL & (1 << 16)));   /* COUNTFLAG */
SysTick->CTRL = 0;
```

COUNTFLAG (bit 16) is set every time the counter reaches zero and is **cleared by reading CTRL**, so the inner loop consumes exactly one millisecond per pass. Because there is no interrupt there is no `SysTick_Handler`, no `volatile` global and no chance of forgetting the handler name – three fewer things to get wrong under exam pressure.

The press length is then measured by counting those milliseconds while the button is held: `while (!(GPIOC->IDR & (1<<13))) { delay_ms(10); t += 10; }`, giving a resolution of 10 ms, which is far finer than the 1000 ms decision threshold.

5.4 Register configuration summary

Register	Value written	Why
`RCC->APB1ENR`	`(1<<21)`	I2C1 clock
`GPIOB->OTYPER`	`(1<<8) | (1<<9)`	**open drain** – the defining setting of an I2C pin
`GPIOB->AFR[1]`	`(4<<0) | (4<<4)`	AF4 = I2C1 on PB8 and PB9
`I2C1->CR2`	`16`	FREQ = APB1 clock in MHz
`I2C1->CCR`	`80`	100 kHz standard mode (master only)
`I2C1->TRISE`	`17`	1000 ns maximum rise time
`I2C1->OAR1` (slave)	`(0x28<<1) | (1<<14)`	own 7-bit address; bit 14 must be written as 1
`I2C1->CR1`	`1`, then `(1<<10)`	PE first, ACK second – the order matters
`I2C1->CR1`	`(1<<8)` / `(1<<9)`	START / STOP
`I2C1->SR1`	bits 0 `SB`, 1 `ADDR`, 2 `BTF`, 6 `RXNE`, 7 `TXE`	start sent, address matched, byte transfer finished, data flow
`I2C1->SR2`	read to clear ADDR	also carries BUSY and the direction bit TRA

5.3 Minimum register sequence – memorise this order

MASTER	SLAVE
1. `RCC->AHB1ENR`: GPIOA, GPIOB, GPIOC	1. `RCC->AHB1ENR`: GPIOA, GPIOB
2. `RCC->APB1ENR`: I2C1	2. `RCC->APB1ENR`: I2C1
3. PC13 pull-up, PA5 output	3. PA5 output
4. PB8, PB9 → alternate function + **open drain**, AFRH = 4	4. PB8, PB9 → alternate function + **open drain**, AFRH = 4
5. `CR2 = 16`, `CCR = 80`, `TRISE = 17`	5. `CR2 = 16`, `OAR1 = (addr<<1) | (1<<14)`
6. `CR1 = PE`	6. `CR1 = PE`, **then** `CR1 |= ACK`
7. send: START, SB, address, ADDR, read SR2, TXE, data, BTF, STOP	7. loop: ADDR, read SR1+SR2, RXNE, read DR, blink, clear STOPF

6 Signal timing

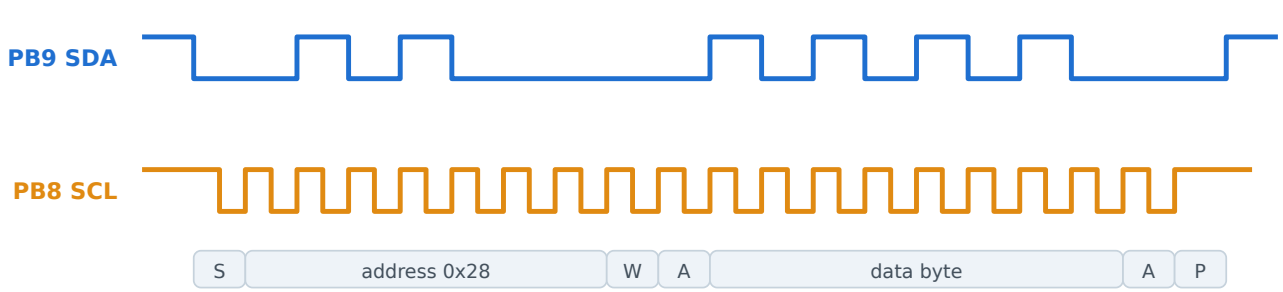


Figure 2 – A write transaction. Both devices can only pull the lines down; the rise back to 3.3 V is done by the pull-up resistors, which is why their value sets the maximum bus speed.

7 MASTER source code i2c_master.c

Type this over the whole contents of `Core/Src/main.c` in the master project. It is self-contained: no other file to add, no header to edit. The comment block at the top is for you – skip it if you are short of time.

i2c_master.c

```c
/* EXP 4  I2C1 (100 kHz, slave address 0x28)  --- MASTER --- NUCLEO-F446RE
 * Wiring : PB8 SCL (CN10-3 / D15) <-> slave PB8   + 4.7k to +3V3
 *          PB9 SDA (CN10-5 / D14) <-> slave PB9   + 4.7k to +3V3
 *          GND     (CN10-20)      <-> slave GND
 * The pull-ups are NOT optional: both pins are open drain.            */

#include "stm32f446xx.h"

#define WAIT(c) { uint32_t w = 200000; while (!(c) && --w); }

static void delay_ms(uint32_t ms)
{
    SysTick->LOAD = 16000 - 1;
    SysTick->VAL  = 0;
    SysTick->CTRL = 5;
    while (ms--) while (!(SysTick->CTRL & (1 << 16)));
    SysTick->CTRL = 0;
}

static void i2c_send(uint8_t b)
{
    I2C1->CR1 |= (1 << 8);                    /* START                */
    WAIT(I2C1->SR1 & (1 << 0));               /* SB                   */
    I2C1->DR = 0x28 << 1;                     /* address + write bit  */
    WAIT(I2C1->SR1 & (1 << 1));               /* ADDR                 */
    (void)I2C1->SR2;                          /* SR1+SR2 clears ADDR  */
    WAIT(I2C1->SR1 & (1 << 7));               /* TXE                  */
    I2C1->DR = b;
    WAIT(I2C1->SR1 & (1 << 2));               /* BTF                  */
    I2C1->CR1 |= (1 << 9);                    /* STOP                 */
}

int main(void)
{
    uint32_t t;

    RCC->AHB1ENR |= (1 << 0) | (1 << 1) | (1 << 2);   /* GPIOA, B, C    */
    RCC->APB1ENR |= (1 << 21);                        /* I2C1           */

    GPIOC->PUPDR  |= (1 << 26);               /* PC13 pull-up  (B1)   */
    GPIOA->MODER  |= (1 << 10);               /* PA5 output    (LD2)  */
    GPIOB->MODER  |= (2 << 16) | (2 << 18);   /* PB8, PB9 alternate   */
    GPIOB->OTYPER |= (1 << 8) | (1 << 9);     /* OPEN DRAIN           */
    GPIOB->AFR[1] |= (4 << 0) | (4 << 4);     /* AF4 = I2C1           */

    I2C1->CR2   = 16;                         /* PCLK1 = 16 MHz       */
    I2C1->CCR   = 80;                         /* 16e6 / (2 x 100e3)   */
    I2C1->TRISE = 17;                         /* 1000 ns / 62.5 ns +1 */
    I2C1->CR1   = 1;                          /* PE                   */

    while (1) {
        if (GPIOC->IDR & (1 << 13)) continue;

        t = 0;
        while (!(GPIOC->IDR & (1 << 13))) { delay_ms(10); t += 10; }

        i2c_send((t >= 1000) ? 'L' : 'S');

        GPIOA->ODR |=  (1 << 5);
        delay_ms(100);
        GPIOA->ODR &= ~(1 << 5);
```

```
62    }
63}
```

8 SLAVE source code i2c_slave.c

Type this over the whole contents of `Core/Src/main.c` in the slave project. It is self-contained: no other file to add, no header to edit. The comment block at the top is for you – skip it if you are short of time.

i2c_slave.c

```c
/* EXP 4  I2C1 slave, own address 0x28  ---  SLAVE  ---  NUCLEO-F446RE
 * OAR1 bit 14 must be written as 1, and ACK must be set AFTER PE = 1.
 * A slave programs no CCR: the master supplies SCL.
 * Wiring : PB8 SCL (CN10-3), PB9 SDA (CN10-5), GND (CN10-20)
 *          + one 4.7k pull-up on each line to +3V3 (CN7-16).          */

#include "stm32f446xx.h"

#define WAIT(c) { uint32_t w = 200000; while (!(c) && --w); }

static void delay_ms(uint32_t ms)
{
    SysTick->LOAD = 16000 - 1;
    SysTick->VAL  = 0;
    SysTick->CTRL = 5;
    while (ms--) while (!(SysTick->CTRL & (1 << 16)));
    SysTick->CTRL = 0;
}

static void blink(uint32_t d)
{
    for (int i = 0; i < 3; i++) {
        GPIOA->ODR |=  (1 << 5);  delay_ms(d);
        GPIOA->ODR &= ~(1 << 5);  delay_ms(d);
    }
}

int main(void)
{
    uint8_t c;

    RCC->AHB1ENR |= (1 << 0) | (1 << 1);        /* GPIOA, GPIOB         */
    RCC->APB1ENR |= (1 << 21);                  /* I2C1                 */

    GPIOA->MODER  |= (1 << 10);                 /* PA5 output    (LD2)  */
    GPIOB->MODER  |= (2 << 16) | (2 << 18);     /* PB8, PB9 alternate   */
    GPIOB->OTYPER |= (1 << 8) | (1 << 9);       /* OPEN DRAIN           */
    GPIOB->AFR[1] |= (4 << 0) | (4 << 4);       /* AF4 = I2C1           */

    I2C1->CR2  = 16;                            /* PCLK1 = 16 MHz       */
    I2C1->OAR1 = (0x28 << 1) | (1 << 14);       /* own address, bit14=1 */
    I2C1->CR1  = 1;                             /* PE                   */
    I2C1->CR1 |= (1 << 10);                     /* ACK, only after PE   */

    while (1) {
        if (!(I2C1->SR1 & (1 << 1))) continue;  /* ADDR = we are called */
        (void)I2C1->SR1;
        (void)I2C1->SR2;                        /* clears ADDR          */

        WAIT(I2C1->SR1 & (1 << 6));             /* RXNE                 */
        c = (uint8_t)I2C1->DR;

        blink(c == 'L' ? 1000 : 150);

        (void)I2C1->SR1;
        I2C1->CR1 |= 1;                         /* clears STOPF         */
    }
}
```

9 Building it in STM32CubeIDE without touching CubeMX

1. **File → New → STM32 Project.** Open the *Board Selector* tab, type `NUCLEO-F446RE`, select it and press *Next*.
2. Name the project, keep *Targeted Language* = **C** and *Targeted Binary Type* = **Executable**, and set ***Targeted Project Type* = Empty**. This is the step that matters: an *Empty* project gives you the startup file, the linker script, `system_stm32f4xx.c` and the CMSIS register headers, but **no HAL, no LL drivers and no .ioc file**. There is nothing to click in CubeMX because there is no CubeMX.
3. If a dialog offers to initialise all peripherals, answer **No** – with an *Empty* project it normally does not appear at all.
4. Open `Core/Src/main.c`, select everything, delete it, and type the listing from this manual. `#include "stm32f446xx.h"` resolves through `Drivers/CMSIS/Device/ST/STM32F4xx/Include`, which the wizard already placed on the include path. There is no second file to create.
5. Build with **Ctrl + B**, connect the board, then *Run → Run As → STM32 C/C++ Application*.
6. Repeat for the second project with the other listing and flash the second board. Mark the boards **MASTER** and **SLAVE** – once they are off the PC they look identical.

> **If the build complains**
>
> `undefined reference to SystemInit` means the project was created without `system_stm32f4xx.c`; add `void SystemInit(void) { }` at the top of `main.c` – but only in that case, otherwise you get a duplicate symbol. `unknown type name 'uint32_t'` means the device header was not found: check that `STM32F446xx` is listed under *Project → Properties → C/C++ Build → Settings → MCU GCC Compiler → Preprocessor*.

10 Running the pair with no PC attached

Nothing in these programs needs the debugger. As soon as the board is powered, the reset vector runs `main()` from flash, so the pair works on its own.

Option A – one USB supply per board (recommended)

1. Unplug both boards from the PC after flashing.
2. Plug each board's CN1 Mini-B socket into a USB power bank, a 5 V phone charger or a powered hub. Leave jumper **JP5 on U5V**, its factory position.
3. LD3 (red) lights on each board. Keep the protocol wires and the common ground exactly as they were during programming.

Option B – one 5 V source for both boards

1. Move **JP5 to E5V** on *both* boards.
2. Feed +5 V into `E5V` (CN7 pin 6) and 0 V into `GND` (CN7 pin 8 or CN10 pin 20) on both boards from the same supply.
3. The grounds are then already common, so the separate GND jumper becomes redundant – leaving it in place does no harm.

> **Safety**
>
> Never drive 5 V into the `+5V` pin while JP5 is on U5V *and* a USB cable is plugged in – that connects two supplies together. Pick one scheme and stay with it. BOOT0 must stay low (its default) so the MCU boots from flash.

11 Test procedure and observations

1. Flash the master listing to board 1 and the slave listing to board 2.
2. Power both boards down, then wire them as in the connection table. Check the ground wire twice – a missing ground is the single most common cause of a dead link.
3. Power both boards. LD3 (red) must be lit on each.
4. **Test 1 – short click.** Press and release B1 on the master in well under a second. The master's LD2 gives one short flash and the slave answers with three fast blinks.
5. **Test 2 – long press.** Hold B1 for at least one second, then release. The slave now blinks three times with one second on and one second off – two seconds from the start of one blink to the start of the next.
6. Time the slow pattern with a stopwatch. Repeat each test three times and record it.
7. **Test 3 – boundary.** Try a press of about one second and confirm which pattern you get. The threshold is the `t >= 1000` comparison in the master.
8. If a scope or logic analyser is available, capture the line(s) during one press and compare with the timing figure in section 6.

Trial	Press duration (ms)	Byte expected	Master LD2 flashed	Blinks counted	Measured spacing (s)	Pass
1						
2						
3						
4						
5						
6						

12 Troubleshooting

Symptom	Cause and remedy
Nothing happens and a scope shows SCL stuck low	Either the pull-up resistors are missing, or ADDR was never cleared and the peripheral is stretching the clock. Fit the resistors and reset both boards.
The master's LD2 flashes but the slave does nothing	The addresses do not match, or `OAR1` bit 14 was not set. The master sends `0x28 << 1`; the slave must store the same `0x28 << 1` plus bit 14.
It works once and then the slave goes deaf	`STOPF` was not cleared. The slave loop must end with `(void)I2C1->SR1; I2C1->CR1 |= 1;`.
SDA and SCL swapped	PB8 is SCL and PB9 is SDA. Swapped, you get no acknowledge at all.
The slave never reacts	Ground is not common. Join GND (CN10-20) of the two boards – this is by far the most frequent fault.
The master's LD2 does not flash either	The master board is not running, or B1 was not detected. Check LD3 (red) is lit and that you typed `GPIOC->PUPDR |= (1 << 26);` – without the pull-up, PC13 floats and never reads as pressed.
Blink timing is wrong (much faster or slower than 2 s)	SYSCLK is not 16 MHz. Make sure no `SystemClock_Config()` or PLL setup was left in `main.c`, and that `SysTick->LOAD` is 16000 − 1.
Both boards blink but the roles are swapped	The master listing was flashed to the slave board. Label the boards.
A long press is read as a short one	You released before 1000 ms. The threshold is the literal `1000` in the master's `t >= 1000` test – change it there if your lab asks for a different value.

13 Review questions

1. Why must I2C pins be open drain, and what does that imply for the pull-ups?
2. How is a START condition different from an ordinary data transition?
3. How does a slave acknowledge, and what happens if it does not?
4. Compute CCR and TRISE for 400 kHz fast mode at PCLK1 = 16 MHz.
5. Why must SR1 and SR2 be read in that order to clear ADDR?
6. Why must ACK be set after PE rather than with it?
7. What is clock stretching, and which side does it?
8. Compare the wire count and throughput of I2C and SPI for this same task.